README

WHU OS Lab Skeleton — 渐进式内核框架

本框架是一个”最小化但完整”的操作系统内核骨架，供学生在8个实验中依次填充实现。

设计原则

- **每个实验只新增文件/功能**，不修改前面实验已完成的部分
- **框架可编译**，但运行时无输出（直到你填完TODO）
- **TODO注释**精确标注每个需要填写的位置

实验路线图

实验	新增文件	验收效果

Lab1	`kernel.ld`
	`kernel/boot/entry.S`
	`kernel/driver/uart.c`
	`kernel/boot/main.c`
	`Makefile`
		屏幕输出
		`Hello OS from RISC-V Bare-metal!`

Lab2	`kernel/driver/console.c`	屏幕输出格式化信息，支持 `%d %x %s %p`

Lab3	`kernel/kalloc.c`
	`kernel/vm.c`
	`kernel/include/riscv.h`
	(扩充)
		分页开启，内核在虚拟内存下继续运行

Lab4	`kernel/boot/start.c`
	`kernel/trap/kernelvec.S`
	`kernel/trap/trap.c`
		每隔1秒屏幕打印一次
		`Tick!`

Lab5	`kernel/proc/proc.h`
	`kernel/proc/proc.c`
	`kernel/proc/swtch.S`
		两个任务交替打印各自的信息

Lab6	`user/usys.S`
	`kernel/syscall/syscall.c`
	`kernel/syscall/sysproc.c`
		用户程序调用
		`getpid()`
		并打印结果

Lab7	`kernel/fs/bio.c` `kernel/fs/fs.c`	创建文件并写入内容，重启后内容保留
Lab8	自由扩展	综合项目

目录结构

```
whu-oslab-skeleton/
├── Makefile                    # 构建系统（Lab1完成后可用）
├── kernel.ld                   # 链接脚本（Lab1任务1）
├── kernel/
│   ├── include/
│   │   ├── types.h             # 基础类型定义（已提供）
│   │   ├── param.h             # 系统参数（已提供）
│   │   ├── riscv.h             # RISC-V寄存器操作（已提供）
│   │   ├── defs.h              # 函数声明汇总（随实验逐步扩展）
│   │   ├── memlayout.h         # 内存布局常量（已提供）
│   │   └── proc.h              # 进程结构（Lab5新增）
│   ├── boot/
│   │   ├── entry.S             # 引导汇编（Lab1任务2）
│   │   ├── start.c             # M-Mode初始化（Lab4新增）
│   │   └── main.c              # 内核主函数（Lab1任务4，后续扩展）
│   ├── driver/
│   │   ├── uart.c              # 串口驱动（Lab1任务3）
│   │   └── console.c           # printf实现（Lab2新增）
│   ├── trap/
│   │   ├── kernelvec.S         # 内核陷阱入口汇编（Lab4任务2）
│   │   └── trap.c              # 陷阱分发（Lab4任务3，Lab6修改）
│   ├── mm/
│   │   ├── kalloc.c            # 物理内存分配器（Lab3任务1）
│   │   └── vm.c                # 虚拟内存/页表（Lab3任务2-4）
│   ├── proc/
│   │   ├── proc.c              # 进程管理（Lab5任务1,3,4）
│   │   └── swtch.S             # 上下文切换汇编（Lab5任务2）
│   ├── syscall/
│   │   ├── syscall.c           # 系统调用分发（Lab6任务3）
│   │   └── sysproc.c           # 系统调用实现（Lab6任务4）
│   └── fs/
│       ├── bio.c               # 块缓冲层（Lab7任务1）
│       └── fs.c                # 文件系统核心（Lab7任务2-3）
└── user/
    ├── usys.S                  # 用户态系统调用桩（Lab6任务1）
    └── init.c                  # 第一个用户程序（Lab6提供）
```

如何开始

```
# 每个实验开始时，将对应的 TODO 填充完整
# 然后运行：
make run
```

验收标准：`make run` 后出现正确输出即为通过。